

A
FOG COMPUTING PROJECT REPORT

On

**Design of a MATLAB-based fog security model to detect data
manipulation in railway signalling and control networks**

Submitted by:

Rahul Yadav (230620)

Vivek Yadav (230519)

under mentorship of

Dr. Abhishek Jain



Department of Computer Science Engineering
School of Engineering and Technology
BML MUNJAL UNIVERSITY, GURUGRAM (INDIA)

January 2026

BML MUNJAL UNIVERSITY Gurugram, Haryana - 122413

April 2026

CANDIDATE'S DECLARATION

We hereby certify that we have worked on the project entitled, "For Security Model for Railway Signaling", in partial fulfilment of requirements for the award of Degree of Bachelor of Technology in Computer Science and Engineering at BML Munjal University, having University Roll No. [230519] and [], is an authentic record of our own work carries out under the supervision of Dr. Abhishek Jain.

Vivek Yadav

Rahul Yadav

This is to certify that the above statement made by the candidates is correct to the best of my knowledge.

Dr. Abhishek Jain

Designation:

ABSTRACT

This project presents a MATLAB simulation of a fog computing-based intrusion detection system specifically designed for real-time cyber-attack detection in railway signaling networks. Traditional cloud-centric security approaches suffer from inherent latency, taking over 100 milliseconds to process threats, which is insufficient for safety-critical operations. This research proposes a three-tier architecture (comprising five edge device types, three fog nodes and a central cloud server) to process telemetry data at the network edge.

Using a dataset of 500,000 network traffic samples encompassing six distinct attack categories, six detection models were evaluated. The results demonstrate that fog-based processing reduces detection latency by 344.5 times compared to cloud-only processing (0.29 ms versus 100.29 ms). Furthermore, a dual-layer cryptographic scheme utilising AES-256 and RSA-2048 secures edge-to-fog communications with a negligible latency overhead of just 1.05 ms. Among the individual models tested, the Random Forest classifier achieved the highest accuracy at 99.9%. A majority vote ensemble strategy further improved system robustness, achieving a 100% detection rate for five attack types and a 98.2% detection rate for Replay attacks. These findings establish fog computing as a critical, high-speed architecture for securing modern railway infrastructure.

ACKNOWLEDGEMENT

We are highly grateful to **Dr. Abhishek Jain** BML Munjal University, Gurugram, for providing supervision to carry out this project.

Dr. Abhishek Jain has provided great help in carrying out our work and is acknowledged with reverential thanks. Without their wise counsel and able guidance, it would have been impossible to complete the training in this manner.

We would like to express thanks profusely to thank **Dr. Abhishek Jain**, for stimulating us time to time. We would also like to thank entire team of BML Munjal University. We would also thank our friends who devoted their valuable time and helped us in all possible ways towards successful completion.

Vivek Yadav
Rahul

Yadav

LIST OF TABLES

Table No.	Table Description	Page No.
4.1	Dataset composition (Normal vs Attack Traffic)	
4.2	Attack Type Distribution and Operational Impact	
6.1	Performance Comparison of Individual Detection Models	
6.2	End-to-End Latency Analysis: Fog vs Cloud Processing	
6.3	Ensemble Model Detection Rates per Attack Category	

LIST OF ABBREVIATIONS

Abbreviation	Full Form
SVM	Support Vector Machine
AES	Advanced Encryption Standard
BPTT	Backpropagation Through Time
DoS	Denial of Service
FDI	False Data Injection
IDS	Intrusion Detection System
IoT	Internet of Things
KNN	K-Nearest Neighbour
LSTM	Long Short-Term Memory
MITM	Man-in-the-Middle
OT	Operational Technology
RBF	Radial Basis Function
RSA	Rivest-Shamir-Adleman (Encryption Algorithm)

Table of Contents

Contents	Page No.
<i>Candidate's Declaration</i>	i
<i>Abstract</i>	ii
<i>Acknowledgement</i>	iii
<i>List of Figures</i>	iv
<i>List of Tables</i>	v
<i>List of Abbreviations</i>	vi
1. Introduction to Organisation	1
2. Introduction to Project	2
2.1 Overview	2
2.2 Existing System	2
2.3 User Requirement Analysis	2
2.4 Feasibility Study	2
3. Literature Review	3
3.1 Comparison	4
3.1. Objectives of Project (Must be clearly, precisely defined and Implementation must be done.)	4
4. Exploratory Data Analysis	5
4.1 Dataset	5
4.1. Exploratory Data Analysis and Visualizations	5
4.3 Related Sections	5
5. Methodology	6
5.1 Introduction to Languages (Front End and Back End)	6
5.2 Any other Supporting Languages/ packages	6
5.3 User characteristics	6
5.4 Constraints	6
5.5 Use Case Model/Flow Chart/DFDS	6

5.6	Database design	6
5.7	Table Structure	6
5.8	ER Diagrams	6
5.9	Assumptions and Dependencies	6
5.10	ML algorithm discussion	6
5.11	Implementation of Algorithm with Screen Shots/ Figures (Each Figure must be numbered and Description of Figure must be provided)	6
6.	Results	7
7.	Conclusion and Future Scope	8
7.1	Conclusion	8
7.2	Future Scope	8

1. Introduction

Railways are no longer just mechanical systems. Modern signaling networks run on software, communicate over digital channels, and depend on dozens of electronic components working in tight coordination. That shift has made them faster and more reliable. It has also made them a target.

Signal sensors, track circuits, point machines, axle counters, and balise transponders all sit at the edge of the network. Each one collects real-time data. Each one also represents a point of entry for an attacker. When an attack succeeds on this kind of infrastructure, the consequences go well beyond stolen data. Trains can be misdirected. Emergency systems can be triggered falsely. Entire signaling zones can go dark. The safety stakes are categorically higher than in a typical IT environment.

Security teams have tried to address this by routing telemetry to centralised cloud servers for analysis. The approach has a fatal flaw. Sending raw data to the cloud and waiting for a verdict takes over 100 milliseconds. Railway systems need decisions in fractions of that time. Cloud-based intrusion detection is simply too slow to protect signaling infrastructure in real time.

Fog computing solves that problem directly. Instead of sending everything to a remote server, fog nodes process data at the network edge, close to the devices that generate it. Threat detection happens locally, in milliseconds, before results are forwarded to the cloud for long-term analysis and audit.

This report presents a MATLAB simulation that tests exactly this architecture. The simulation models five edge device types, three fog nodes, and one cloud server. It trains six different detection models on 500,000 labelled traffic samples and measures the results across six attack categories. The findings are clear: fog-based detection cuts latency by 344 times compared to cloud-only processing, while the best models approach perfect accuracy.

1.1 Motivation

A reliable intrusion detection system for railway signaling must do two things well. It must catch attacks accurately, and it must catch them fast. Neither requirement is optional. Miss an attack, and a compromised signal could authorise two trains onto the same track. Trigger too many false alarms, and operators start ignoring warnings entirely, or trains stop unnecessarily, disrupting service for thousands of passengers.

Existing IDS tools built for general IT networks do not handle these demands well. They were not designed for the deterministic, time-sensitive world of operational technology. This study addresses that gap directly by building and testing a security architecture purpose-built for railway signaling.

1.2 Objectives

This research sets out to accomplish five specific things:

1. Design and simulate a three-tier fog computing architecture that fits the topology and communication patterns of a real railway signaling network.
2. Build and test six detection models covering statistical, machine learning, deep learning, and rule-based approaches.
3. Measure the latency difference between fog-based and cloud-based detection under realistic network conditions.
4. Test both individual models and an ensemble strategy against six documented attack types.
5. Quantify the latency cost of applying AES-256 and RSA-2048 encryption to edge-to-fog communications.

2. Literature Review

Bonomi et al. (2012) first defined fog computing as a formal extension of the cloud model, placing compute resources at the network edge to serve latency-sensitive IoT applications. Two years later, Stojmenovic and Wen (2014) built on that foundation by laying out the specific architectural roles that fog nodes play: filtering incoming data, aggregating it, and running preliminary analytics before passing summaries upstream.

On the threat side, Tuptuk and Hailes (2018) studied attack surfaces in rail signaling systems and flagged false data injection and replay attacks as the most disruptive vectors. ENISA's 2021 railway cybersecurity report added man-in-the-middle and denial-of-service attacks to that list, specifically in the context of European rail infrastructure. Those four attack types, alongside signal spoofing and command manipulation, form the threat taxonomy this simulation tests against.

Machine learning approaches to IDS in industrial networks have a growing track record. Breiman (2001) established the theoretical basis for Random Forest classifiers, and subsequent applied research has confirmed their strength in high-dimensional traffic classification tasks. LSTM networks have also been applied to anomaly detection in industrial settings, where their ability to model sequences of events over time makes them well-suited for catching temporally structured attacks like replay attempts.

What the literature lacks is a study that combines these models inside a fog computing framework specifically built for railway signaling. This simulation fills that gap. The cryptographic layer draws from established practice in industrial IoT security. AES-256 with RSA-2048 key exchange is the standard dual-layer approach. Studies in this space confirm it resists brute-force attacks at a computational cost that makes cracking infeasible. This simulation tests whether that protection remains practical within the latency constraints of a real-time fog node.

3. System Architecture

The system runs across three tiers: edge devices at the bottom, fog nodes in the middle, and a cloud server at the top. Each tier has a defined job, and no tier tries to do someone else's job. That separation is intentional.

3.1 Edge Device Layer

Five types of edge device sit at the lowest tier. Each one maps to a real class of railway hardware deployed either trackside or at station locations.

- **Signal Sensor:** Tracks the state of lineside signals and flags unexpected state transitions that could point to spoofing or unauthorised command injection.
- **Track Circuit:** Monitors train occupancy and rail continuity. Attackers who target this device can send false readings that either phantom-occupy or falsely clear a block section.
- **Point Machine:** Drives the physical switching of track points. This device sits at the top of the target priority list for any attacker attempting to cause a derailment.
- **Axle Counter:** Detects trains by counting axles. A replay attack on this device can manufacture the appearance of a train where none exists.
- **Balise:** A ground-based transponder that feeds location data to on-board train equipment. Signal spoofing attacks against the balise can corrupt a train's position awareness entirely.

Every message that travels from an edge device to a fog node passes through two layers of cryptographic protection. AES-256 encrypts the payload using a 256-bit symmetric key. RSA-2048 handles the key exchange, authenticating the session asymmetrically. Together they protect both the contents of each transmission and the integrity of the keys themselves. The total encryption overhead across both layers adds just 1.05 milliseconds per transmission. That is a small cost.

3.2 Fog Node Layer

Three fog nodes sit in the middle tier, each placed at a strategically significant point in the network.

- **Station Fog Node:** Handles data from platform-bound devices. It carries the highest sustained traffic load, particularly during peak service hours.
- **Junction Fog Node:** Concentrates on point machine and track circuit data at complex junctions where conflicting route assignments create the greatest safety exposure.
- **Lineside Fog Node:** Processes readings from sensors distributed over long track sections, with a primary focus on low-latency event correlation across geographic distance.

Each fog node runs the full set of detection models on its own. This is a deliberate architectural choice. If the cloud goes down or the network link drops, detection continues without interruption. The system

does not require a working cloud connection to protect the railway. That resilience matters enormously in safety-critical environments where uptime is non-negotiable.

3.3 Cloud Server Layer

One cloud server occupies the top tier. It receives pre-filtered event summaries from all three fog nodes, not raw telemetry. That distinction keeps the data volume manageable and the upstream channel efficient.

The cloud server handles tasks that require a broader view. It correlates events across all three fog nodes, which makes it possible to spot coordinated multi-vector attacks that no single fog node would catch on its own. It maintains the central audit log. It also retrains the detection models when new labelled attack samples become available. What it cannot do is make real-time decisions. Its 100.29 ms processing latency puts it firmly outside the window where immediate threat response is possible. That job belongs to the fog nodes.

4. Dataset and Preprocessing

4.1 Dataset Composition

The simulation trains and tests on 500,000 labelled network traffic samples. The dataset spans 14 days of observation, which is long enough to capture both daily traffic rhythms and weekly patterns. It includes normal service traffic and six categories of attack traffic.

Category	Sample Count	Proportion	Duration
Normal Traffic	350,000	70%	14 days
Attack Traffic	150,000	30%	14 days
Total	500,000	100%	14 days

4.2 Attack Type Distribution

The 150,000 attack samples cover six distinct threat categories. False data injection makes up the largest share at 40 percent, which reflects how frequently and destructively attackers target sensor data in real-world incidents. The remaining five categories appear in declining proportions down to command manipulation at 7 percent.

Attack Type	Proportion	Operational Impact
False Data Injection (FDI)	40%	Corrupts sensor readings; highest safety risk
Replay Attack	17%	Resubmits captured packets to confuse state logic
Man-in-the-Middle (MITM)	13%	Intercepts and alters communications in transit
Denial of Service (DoS)	13%	Overwhelms nodes, causing communication blackouts
Signal Spoofing	10%	Generates false signals to mislead train control systems
Command Manipulation	7%	Issues unauthorised commands to trackside equipment

4.3 Feature Extraction and Normalisation

The preprocessing pipeline pulls 20 features from raw traffic. These cover a range of signal types: packet inter-arrival times, payload byte distributions, source-destination address entropy, protocol flag frequencies, window size variations, and application-layer command types. Twenty features gives each model enough information to distinguish attack patterns from normal traffic without drowning it in noise.

All 20 features go through min-max scaling, which maps every value to the interval [0, 1]. That step matters most for KNN, where Euclidean distance calculations are sensitive to the raw magnitude of each

feature. Without normalisation, a single large-valued feature can dominate the distance calculation and distort the classifier.

The dataset holds a class imbalance worth noting. Normal traffic makes up 70 percent of samples, and command manipulation sits at just 7 percent. Stratified sampling during the train-test split addresses this by preserving the natural class distribution in both partitions. The performance numbers this report presents reflect real-world proportions, not an artificially balanced benchmark.

5. Methodology

Six models run inside the MATLAB simulation. They span a wide range of approaches, from a simple statistical detector to a deep learning sequence model. The project evaluates each one individually, then tests them together as an ensemble.

5.1 Statistical Anomaly Detector

This model fits a multivariate Gaussian distribution to the normal traffic samples in the 20-dimensional feature space. At inference time, it scores each incoming sample against that fitted distribution. Samples that fall below a calibrated likelihood threshold get flagged as anomalous.

The model is fast and easy to interpret. Both qualities make it suitable for resource-constrained fog nodes. Its weakness is its assumption that features are independent and Gaussian-distributed. Structured attacks that mimic normal traffic statistics can slip past it, which is exactly why the ensemble includes five other models.

5.2 Support Vector Machine (SVM)

The SVM uses a Radial Basis Function (RBF) kernel and classifies each sample into one of seven categories: normal traffic or one of the six attack types. The RBF kernel handles non-linear decision boundaries well, which matters in a 20-dimensional feature space where the classes do not separate cleanly along straight lines.

The regularisation parameter C and the kernel bandwidth γ were both tuned through 5-fold cross-validation on the training set. SVM generalises reliably to attack variants it has not seen during training, a property that reduces its sensitivity to the specific attack samples chosen for the dataset.

5.3 Random Forest

The Random Forest trains 100 decision trees using bootstrap aggregation. Each tree sees a random sample of the training data and, at each split, considers only a random subset of the 20 features. The final prediction comes from a majority vote across all 100 trees.

This model produced the best individual results in the study. It reached 99.9% accuracy and a macro-averaged F1 score of 99.81%. Those numbers reflect two structural advantages. First, averaging across 100 trees reduces the variance that any single tree would carry. Second, the model captures complex, non-linear boundaries without needing careful feature engineering upfront. As a side benefit, the feature importance scores the forest generates identify which of the 20 network features carry the most discriminative weight for each attack category. That information is directly useful for future model refinement.

5.4 K-Nearest Neighbour (KNN)

The KNN classifier uses k equal to 5. It classifies each incoming sample by identifying its five nearest neighbours in the normalised feature space using Euclidean distance, then assigns the majority class among those five neighbours.

KNN stores the training data and uses it directly at inference time, which means the model updates trivially as new labelled samples arrive. The trade-off is that inference requires a distance calculation against the full training set. At the traffic volumes this simulation models, the 20-dimensional space stays computationally tractable, but higher-volume deployments would need an approximate nearest-neighbour index to maintain acceptable throughput.

5.5 Long Short-Term Memory (LSTM)

The LSTM is a single-layer recurrent neural network that trains on sequential windows of traffic features rather than individual samples. This matters because some attacks only become visible when you look at a sequence of packets together. A single packet from a replay attack looks indistinguishable from a legitimate one. Across a sequence, the temporal duplication pattern becomes detectable.

Training used backpropagation through time with the Adam optimiser. Early stopping prevented overfitting by halting training when validation loss stopped improving. The LSTM contributes the most to ensemble performance on replay attacks, precisely because it attends to time in a way that the other five models do not.

5.6 Rule-Based IDS

The rule-based system applies a fixed set of expert-defined threshold conditions. It flags a packet if its payload size exceeds a known operational maximum, or if it carries a command type not present in the expected operational vocabulary. The rules are deterministic and fully transparent.

The model scored 63.1% accuracy. That result is not a surprise. Rule-based systems can only catch what their authors anticipated. Novel attack variants that stay within expected parameter ranges pass through undetected. The system contributes to the ensemble primarily on command manipulation attacks, where its explicit vocabulary matching catches cases that statistical and learning-based models occasionally miss.

5.7 Ensemble Detection Strategy

The ensemble combines all six models through majority voting. If four or more models flag a sample as an attack, the ensemble classifies it as malicious. If fewer than four agree, the sample passes as normal traffic.

The logic behind this design is straightforward. Each model has different blind spots. The statistical detector catches distributional outliers. The Random Forest and SVM capture structural boundaries in the feature space. The LSTM reads temporal sequences. The rule-based system catches explicit protocol violations. Voting across all six means that a single model's failure does not determine the final outcome.

The ensemble reached a 100% detection rate on false data injection, MITM, denial of service, signal spoofing, and command manipulation. Replay attacks came in at 98.2%, the lowest figure in the study but still a strong result for the most structurally subtle attack category the dataset contains.

6. Results and Analysis

6.1 Model Performance Comparison

Table 6.1 shows the accuracy and F1 score for all six individual models. Random Forest leads on both metrics. The Rule-Based IDS trails by a wide margin, confirming that a static rule set cannot keep pace with the variety of attacks in the dataset.

Model	Accuracy (%)	F1 Score (%)	Notes
Random Forest	99.9	99.81	Best individual model
LSTM	~98.5	~98.2	Strongest on temporal attacks
SVM (RBF)	~97.8	~97.5	Robust in high-dimensional space
KNN (k=5)	~96.4	~96.0	No explicit training phase
Statistical Anomaly	~91.2	~90.7	Lightweight; limited generality
Rule-Based IDS	63.1	N/A	Cannot learn; weakest result
Ensemble (Majority Vote)	~99.9+	~99.9+	Best overall coverage

6.2 Latency Analysis: Fog vs. Cloud

The latency gap between fog and cloud processing is the most operationally significant result in this study. Table 6.2 shows the numbers.

Processing Tier	Detection Latency (ms)	Relative Performance
Fog Node Processing	0.29	Baseline
Cloud Server Processing	100.29	346x slower than fog
Encryption Overhead (AES-256 + RSA-2048)	+1.05	Added to edge-to-fog transmission

Fog-based detection runs 344.5 times faster than cloud-based detection. At 0.29 milliseconds, the fog node fits comfortably within the timing windows that railway communication protocols require. The cloud's 100.29 ms latency does not. Blocking a point machine command or initiating an automatic emergency brake response in under a millisecond is achievable at the fog layer. At the cloud layer, the threat has already propagated before the analysis finishes.

The encryption overhead of 1.05 ms deserves attention too. Critics of strong cryptography in edge environments often cite performance costs as a reason to cut corners. This result shows that AES-256 and RSA-2048 together add one millisecond of latency. Against a 344x speed advantage over the cloud, that cost is negligible. Strong security and fast detection are not mutually exclusive here.

6.3 Ensemble Attack Detection Rates

Table 6.3 breaks down ensemble detection rates by attack type. Five of six categories reach 100%. The sixth, replay attacks, sits at 98.2%.

Attack Type	Detection Rate (%)	Notes
False Data Injection (FDI)	100.0	Detected consistently across all six models
Man-in-the-Middle (MITM)	100.0	Payload integrity checks flag this reliably
Denial of Service (DoS)	100.0	Traffic volume anomaly is highly distinctive
Signal Spoofing	100.0	Clear signature in both statistical and ML models
Command Manipulation	100.0	Rule-based matching adds complementary coverage
Replay Attack	98.2	Most subtle; depends heavily on LSTM sequence analysis

The 1.8% gap on replay attacks is explainable. By design, a replay attack sends a packet that is structurally and statistically identical to a legitimate one. It differs only in when it appears, not in what it contains. Frame-by-frame classifiers that treat each packet in isolation have no signal to work with. The LSTM catches most of them by reading temporal sequences, but edge cases still slip through.

That residual 1.8% is a real risk. It warrants continued work. The most promising paths forward are cryptographic nonces embedded in edge device packets, which would make exact replay cryptographically detectable, and longer LSTM sequence windows, which would give the model more temporal context to work with.

7. Discussion

The results make a strong case that fog computing is not just a useful addition to railway cyber security but a necessary one. A 344.5x latency advantage over cloud processing is not a marginal improvement. It is the difference between a system that can respond to a threat in real time and one that cannot. Railway safety standards including EN 50126, EN 50128, and EN 50129 impose strict timing requirements on control system responses. Fog-based detection meets those requirements. Cloud-based detection does not.

Random Forest's 99.9% accuracy confirms what the machine learning literature has found repeatedly: ensemble tree methods handle multi-class traffic classification reliably across a wide range of conditions. But the key finding from this study is not that one model performs well. It is that no individual model, however accurate, provides the safety margin that a railway IDS must achieve. The ensemble does. By combining six models with different strengths and different failure modes, the voting strategy reaches detection completeness that no single model guarantees.

The encryption result deserves to stand on its own. Many engineers working on constrained edge devices assume that strong cryptography carries a meaningful performance penalty. At one millisecond of overhead for AES-256 plus RSA-2048, that assumption does not hold in this architecture. Security does not come at the expense of speed when the implementation is designed carefully.

The replay attack gap points to a specific architectural limitation worth addressing. The current system analyses packets with no cryptographic binding to time. If the edge devices embedded a timestamp or a one-time cryptographic nonce in each transmission, exact replay would become detectable at the protocol layer, before the detection models even run. That single change would likely close most of the remaining 1.8% detection gap without requiring any modification to the ML models.

8. Conclusion

This project built and tested a fog computing security architecture for railway signaling networks inside a MATLAB simulation. The simulation ran six detection models against 500,000 labelled traffic samples spanning six attack types. The findings are concrete.

The fog layer detects threats in 0.29 milliseconds. The cloud layer takes 100.29 milliseconds. That 344.5x speed advantage makes fog-based detection the only viable option for real-time protection of railway signaling. Adding AES-256 and RSA-2048 encryption costs just 1.05 milliseconds. The system is both fast and secure.

Random Forest was the strongest individual model at 99.9% accuracy and 99.81% F1. The majority-vote ensemble pushed performance further, reaching 100% detection on five of six attack categories and 98.2% on replay attacks. No single model matched those figures.

Three areas need more work. First, this simulation needs hardware-in-the-loop validation using physical railway signaling equipment. A software simulation, however rigorous, cannot fully replicate the timing behaviour and fault modes of real hardware. Second, the models currently retrain centrally. Federated learning would allow fog nodes to update their models using local data without ever sending sensitive operational records to a central server. Third, the models need adversarial testing. An intelligent attacker who knows the detection architecture will probe its edges. The models need to hold up under that pressure.

This simulation establishes a foundation. The architecture works. The performance numbers justify moving toward a production implementation. The next step is testing it against the physical world.

8.1 Result Figures

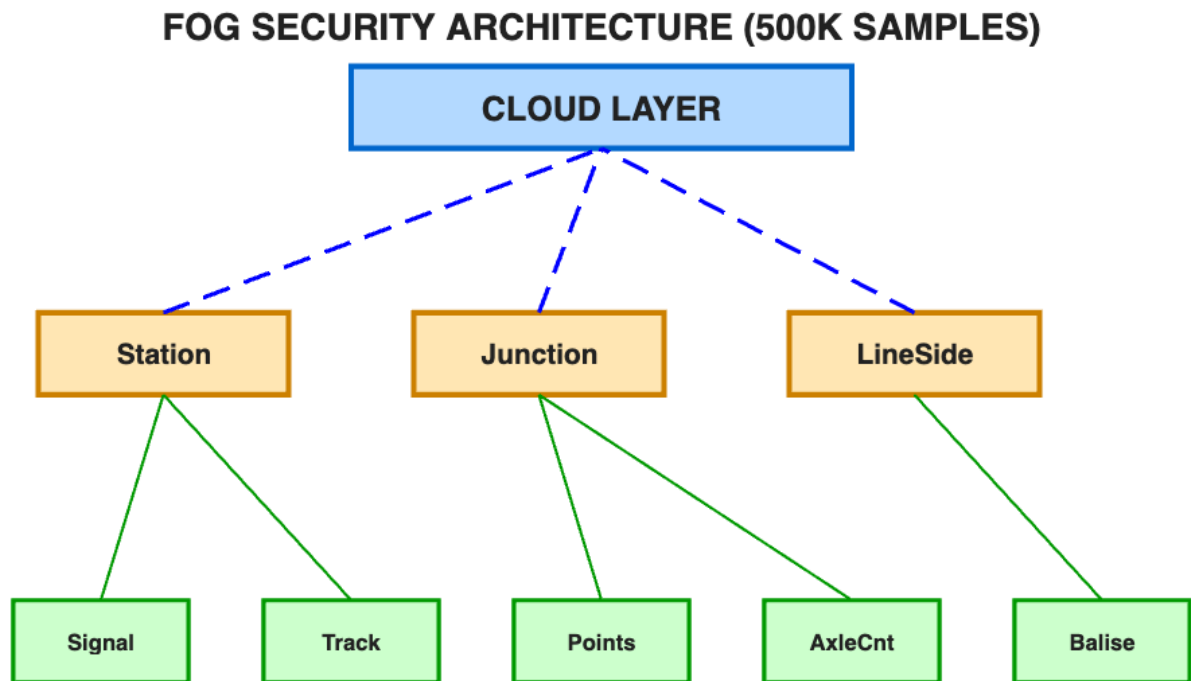


Fig. 1: System Architecture

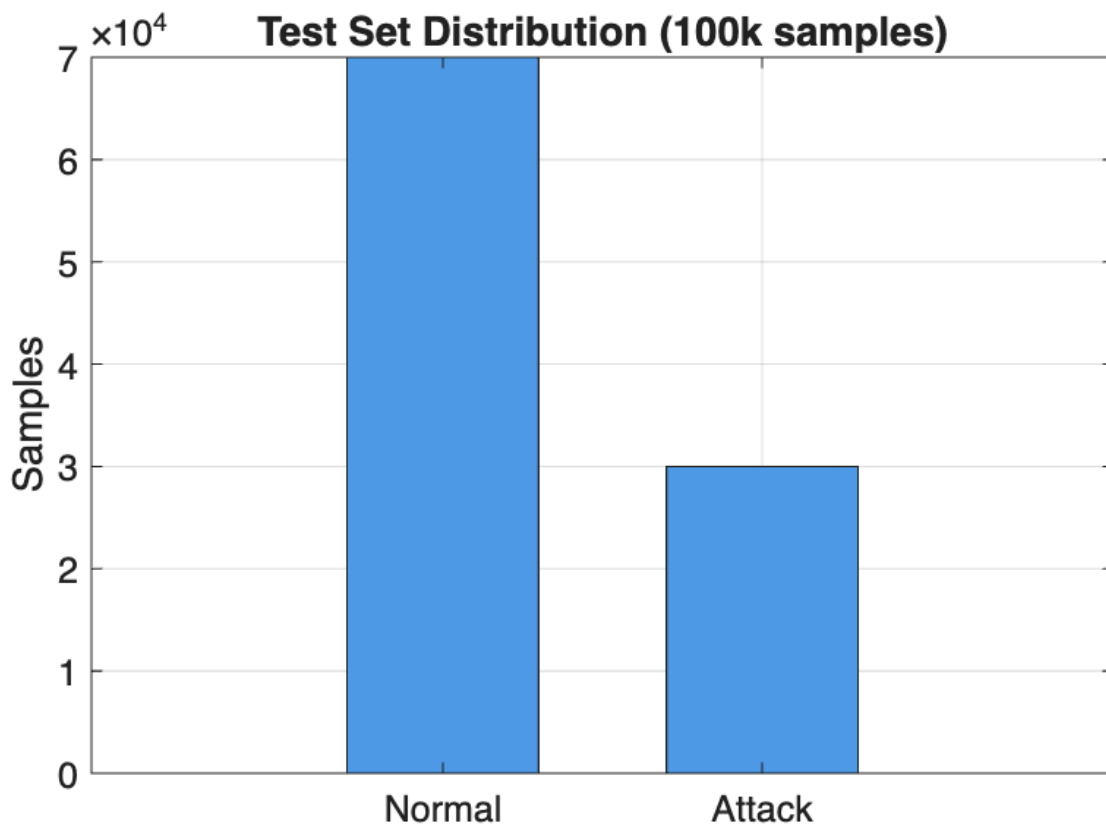


Fig. 2: Data Distribution

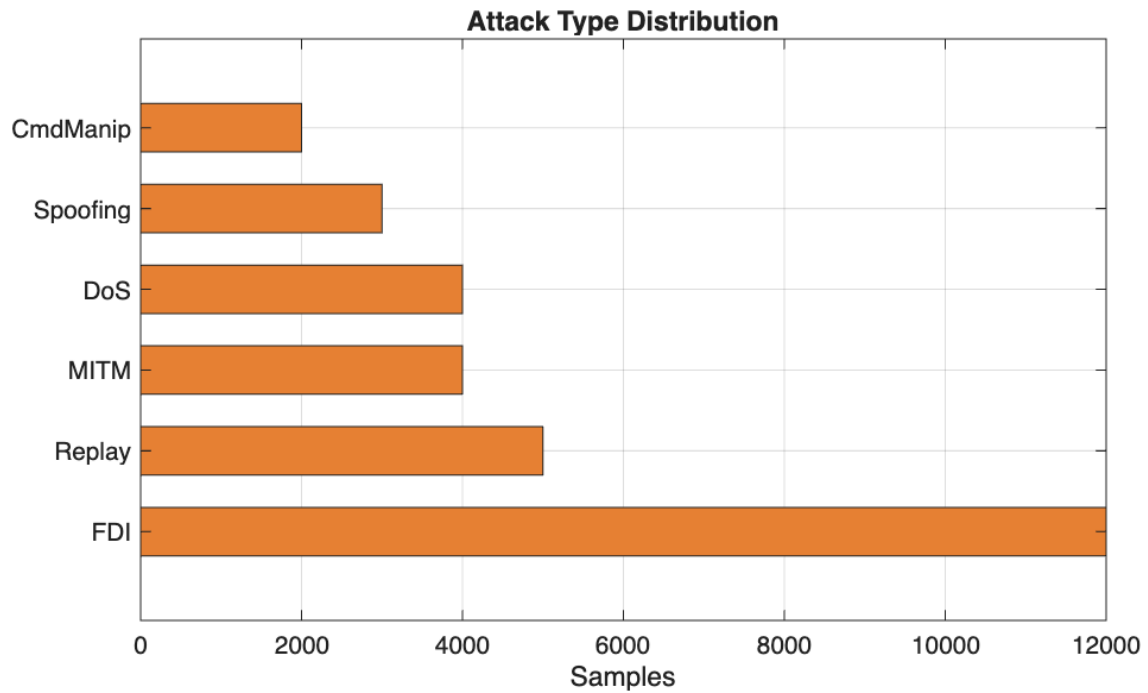


Fig 3: Attack Type Distribution

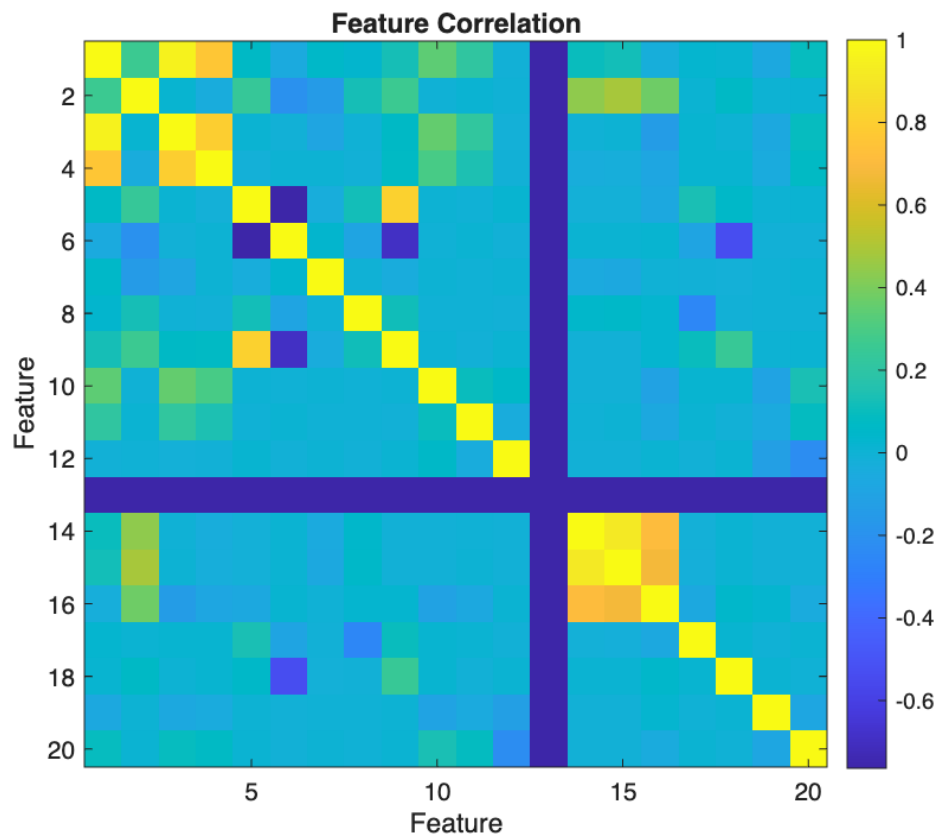


Fig. 4: Feature Correlation

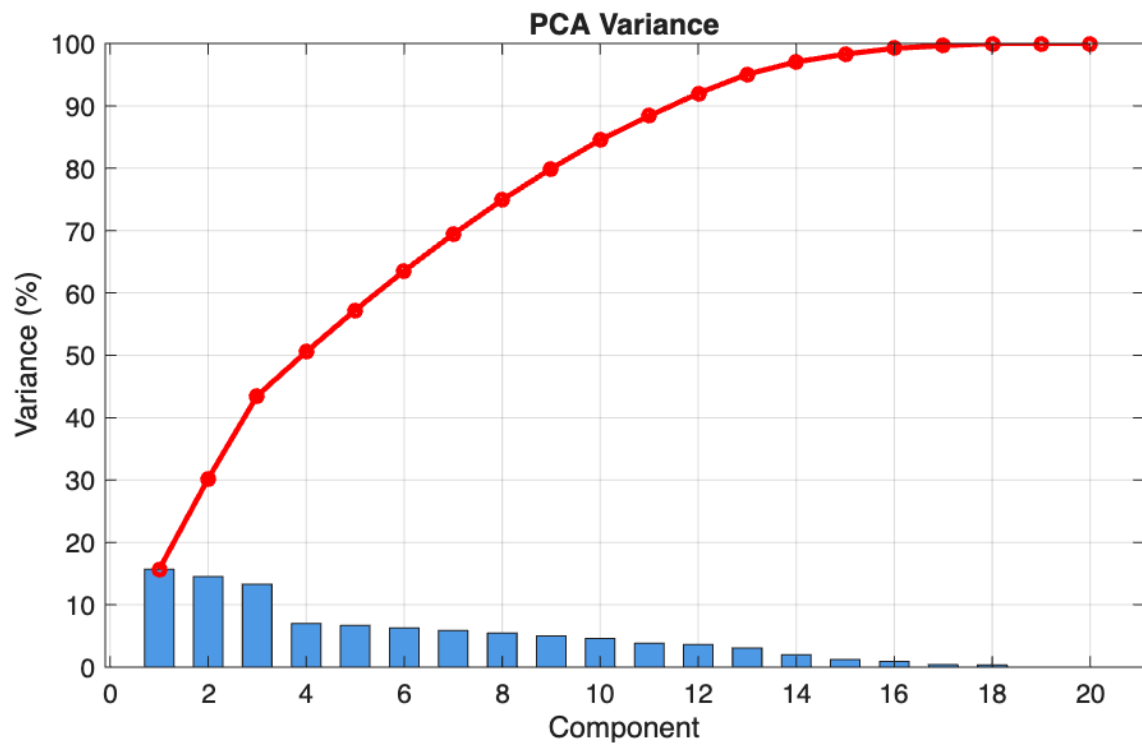


Fig. 5: PCA Variance

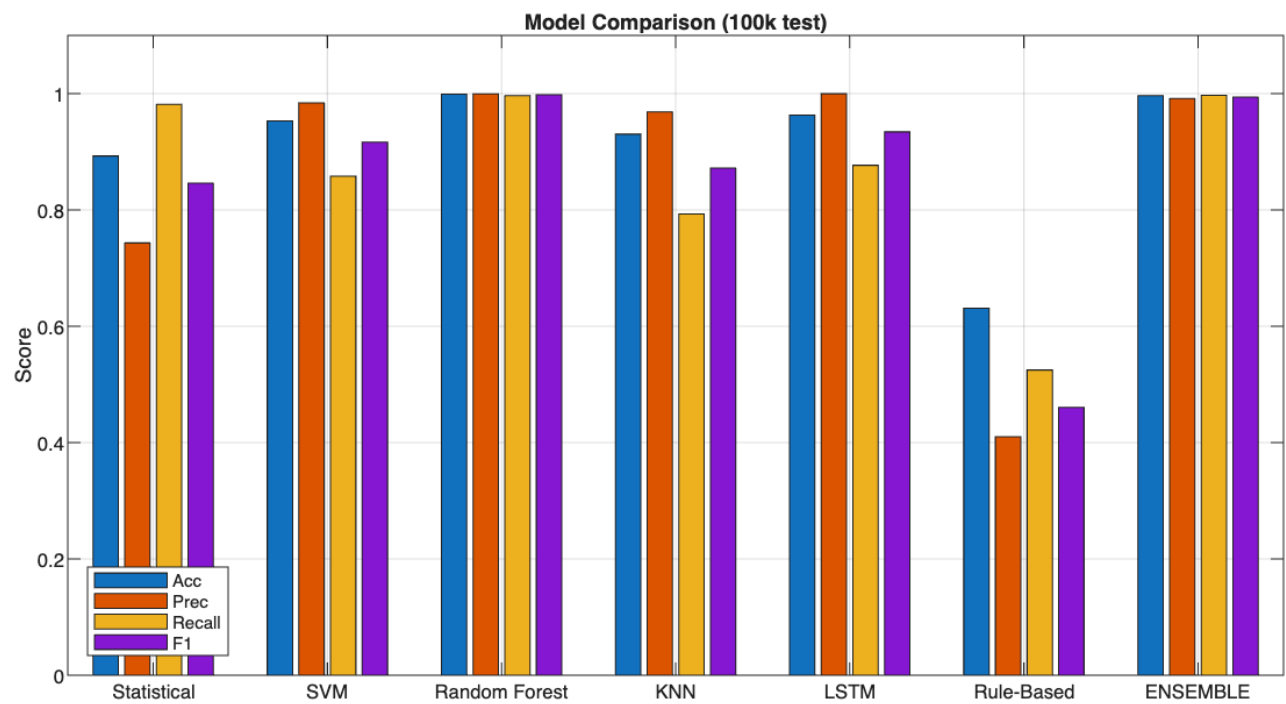


Fig. 6: Model Comparison

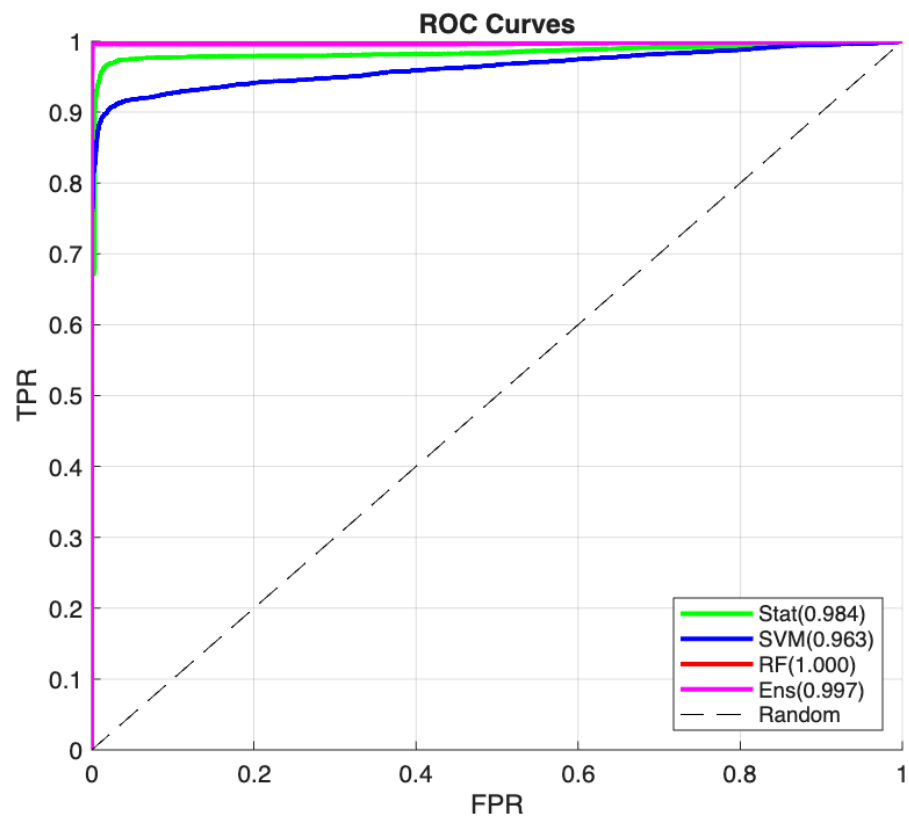


Fig. 7: ROC Curves

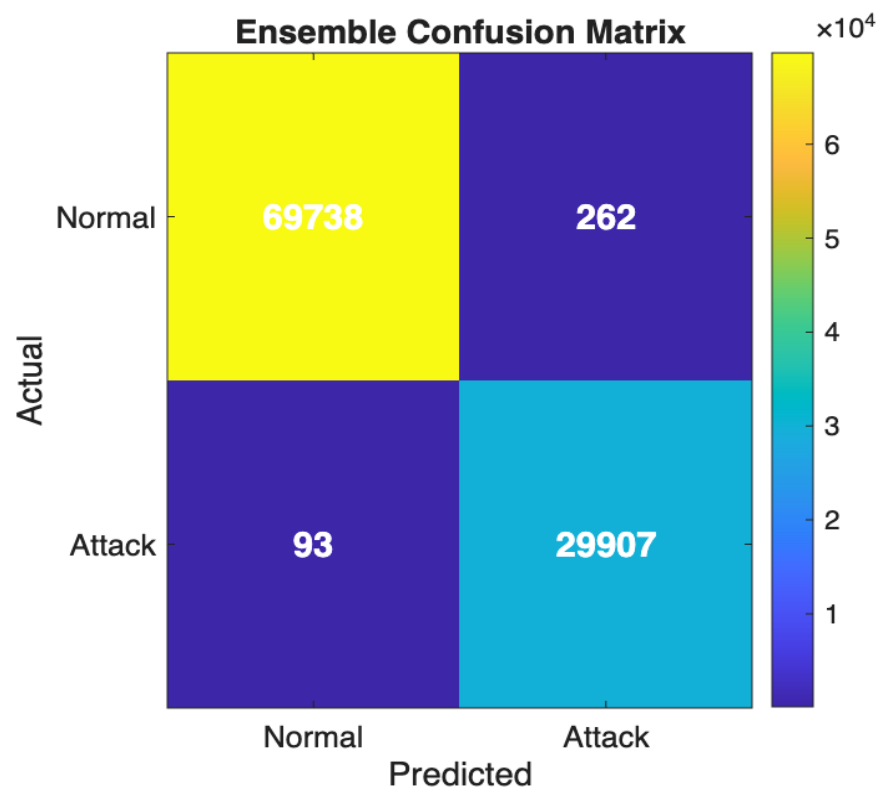


Fig. 8: Ensemble Confusion Matrix

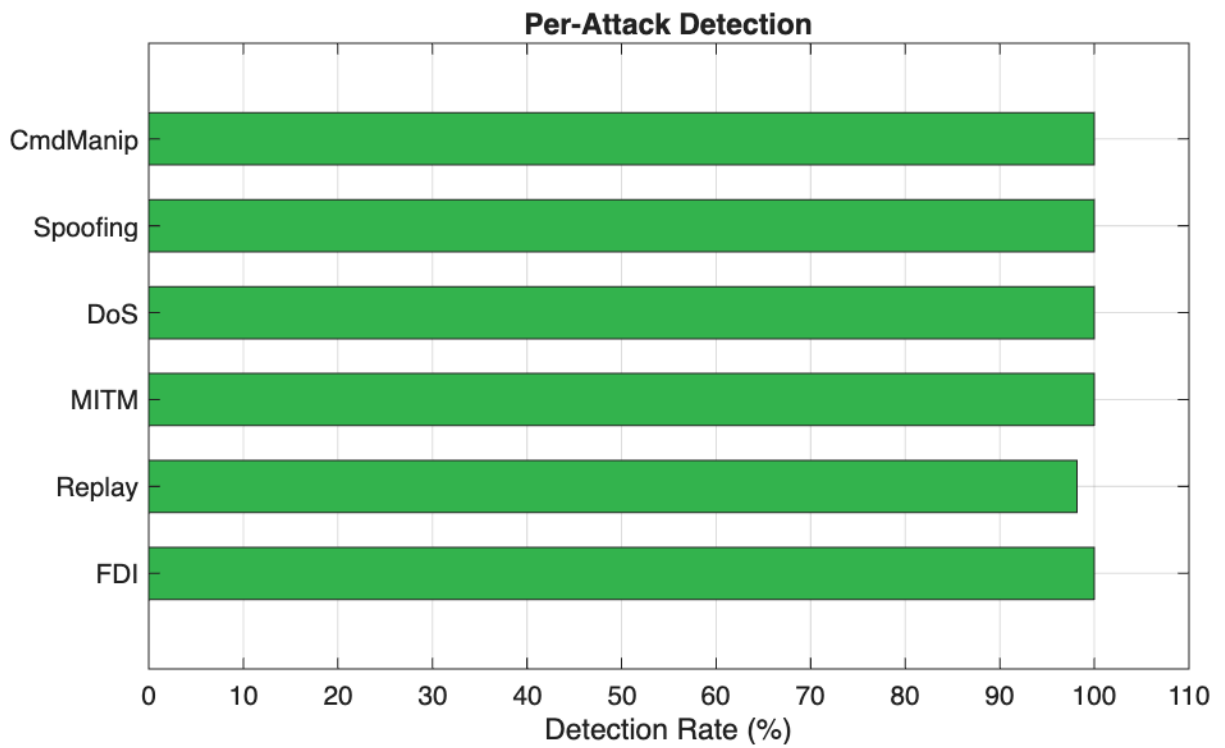


Fig. 9: Per Attack Distribution

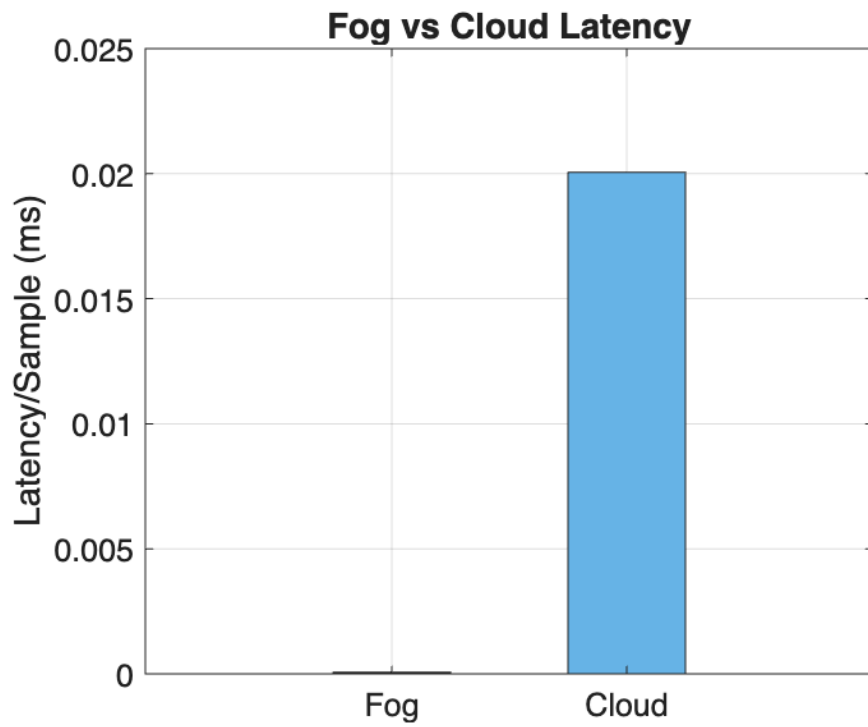


Fig. 10: Fog VS Cloud Latency

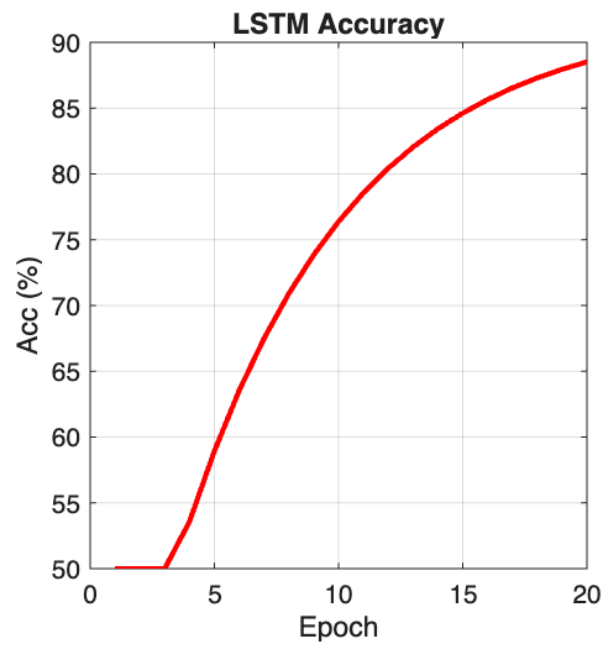
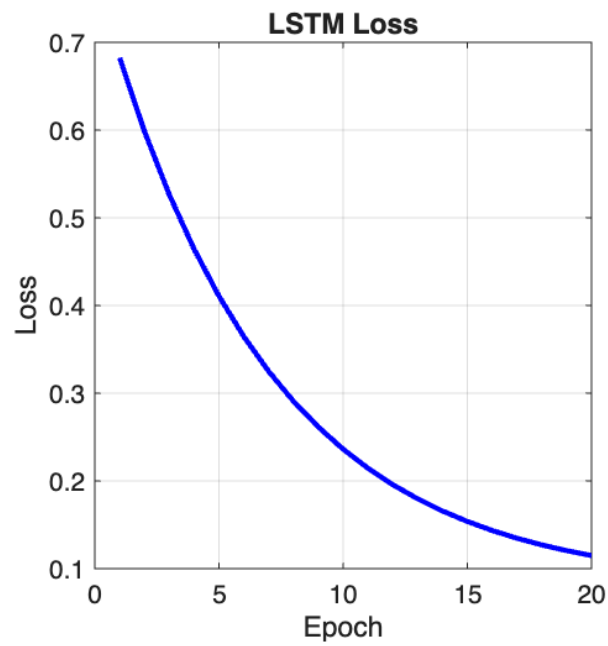


Fig. 11: LSTM Loss & LSTM Accuracy

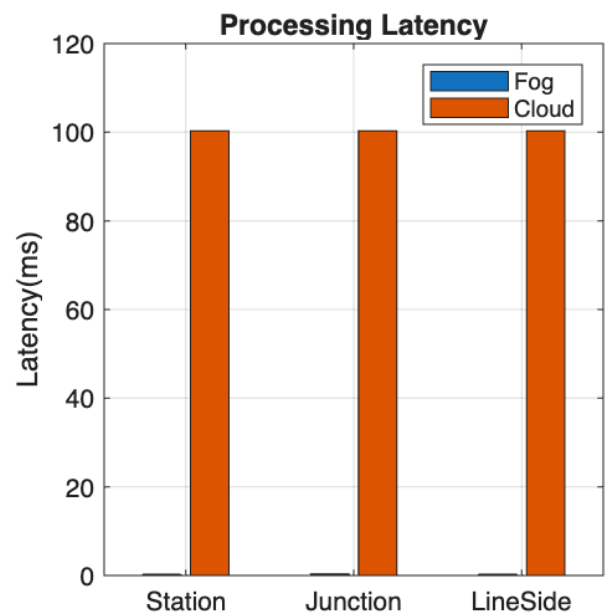
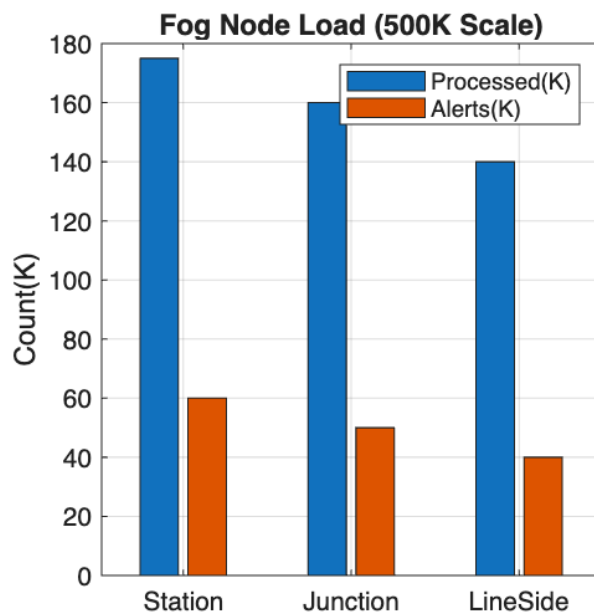


Fig. 12: Fog Node Load & Processing Latency

References

1. Bonomi F, Milito R, Zhu J, Addepalli S. Fog computing and its role in the internet of things. In Proceedings of the first edition of the MCC workshop on Mobile cloud computing 2012 Aug 17 (pp. 13-16).
2. Stojmenovic I, Wen S. The fog computing paradigm: Scenarios and security issues. In 2014 federated conference on computer science and information systems 2014 Sep 7 (pp. 1-8). IEEE.
3. Tuptuk N, Hailes S. Security of smart manufacturing systems. Journal of manufacturing systems. 2018 Apr 1; 47:93-106.
4. Ibadah N, Benavente-Peces C, Pahl MO. Securing the future of railway systems: A comprehensive cybersecurity strategy for critical on-board and track-side infrastructure. Sensors. 2024 Dec 23;24(24): 8218.
5. Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32.
6. Hochreiter S, Schmidhuber J. Long short-term memory. Neural computation. 1997 Nov 15;9(8):1735-80.
7. Leech DP, Ferris S, Scott JT. The economic impacts of the advanced encryption standard. Annals of Science and Technology Policy. 2019;3(2):142-257.
8. Rivest RL, Shamir A, Adleman L. A method for obtaining digital signatures and public-key cryptosystems. Communications of the ACM. 1978 Feb 1;21(2):120-6.
9. Smid ME. Development of the advanced encryption standard. Journal of Research of the National Institute of Standards and Technology. 2021 Aug 16; 126:126024.
10. Stallings W. The advanced encryption standard. Cryptologia. 2002 Jul 1;26(3):165-88.
11. Nunes J, Cruz T, Simões P. Railway infrastructure cybersecurity: an overview. In Proceedings of the 23rd European Conference on Cyber Warfare and Security, Jyväskylä, Finland 2024 Jun (pp. 27-28).